

Construire une application intégrant l'IA

Call To Adventure · Bloc 3 · E4 : 20 minutes

PRODUIT

Une aventure interactive dans le navigateur, avec narration IA et règles de jeu côté serveur.

RÉALISATION

Cadrer, concevoir, organiser et développer les parcours du joueur.

QUALITÉ / LIVRAISON

Tester le métier et les accès.
Automatiser les contrôles.
Préparer un artefact déployable.

Le bloc 3 relie les besoins du joueur aux preuves de fonctionnement.

Permettre de jouer et de retrouver sa progression

Les parcours distinguent le visiteur et le joueur connecté.

VISITEUR

- Choisir une aventure.
- Créer un personnage valide.
- Jouer sans créer de compte.
- Progression liée à la session.

JOUEUR CONNECTÉ

- Retrouver ses sauvegardes.
- Reprendre une partie active.
- Réutiliser ses personnages.
- Gérer ses propres données.

EXPLOITANT / JURY

- Installer et vérifier le service.
- Comprendre les choix techniques.
- Relier exigences, code, tests et démonstration.

La continuité de la partie et la propriété des données orientent la conception.

Transformer le besoin en critères vérifiables

Exemples issus des exigences et des tests du projet.

REPRENDRE SA PARTIE

Un joueur peut charger sa sauvegarde active.
Un autre utilisateur ne peut ni la charger ni la supprimer.

SURVIVRE À UNE PANNE

Un tour IA en échec retourne une erreur contrôlée.
La progression sauvegardée ne doit pas avancer.

UTILISER L'INTERFACE

Navigation et états compréhensibles.
Critères clavier, focus et annonces.
Cible d'accessibilité à valider.

Chaque exigence doit avoir un composant, un test et une preuve de démonstration.

Séparer présentation, métier et persistance

Une application Django avec des services Python internes.

PRÉSENTATION

Templates HTML / CSS / JS



Vues Django et endpoints JSON



Sessions, auth et CSRF

MÉTIER

GameEngine



LangGraph Combat

IA + RAG règles serveur

STOCKAGE / SERVICES

Base Django : comptes / parties.

SQLite catalogue : références.

Chroma / Ollama : contexte.

Fournisseur externe : génération.

Le navigateur envoie des actions ; le serveur décide des mutations autorisées.

Choisir une architecture adaptée au périmètre

Les choix actuels favorisent une démonstration locale reproductible.

DJANGO

Comptes, sessions, ORM et templates dans un même cadre.
Moins de couches à exploiter pour ce projet solo.

SQLITE / SERVICES IA

Données locales de taille modeste.
IA hébergée et retrieval local.
Limites de concurrence et de dépendance externe documentées.

CIBLE DE DÉPLOIEMENT

Conteneur web et proxy HTTPS.
Secrets externes, stockage durable.
Hébergement et SGBD de production à confirmer.

La préproduction cible est décrite ; son accessibilité réelle reste à prouver.

Piloter un projet solo avec des critères explicites

Backlog, priorités, risques et décisions sont documentés.

PRÉPARER

Besoin formulé.
Critères testables.
Composants concernés.
Risques et preuve attendue.
Définition de prêt.

RÉALISER

Sécurité → cadrage → qualité
→ industrialisation → exploitation.
Suivi par éléments de backlog et
commits.

TERMINER / REVOIR

Code et documentation cohérents.
Tests pertinents.
Preuve conservée.
Notes de revue et rétrospective à
maintenir.

Les rituels, dates et échanges réels doivent compléter les supports de pilotage.

Conserver une progression cohérente

Le moteur expose des transitions de jeu et des règles de persistance.

PARCOURS

Accueil → personnage → récit
↓
Choix → combat éventuel
↓
Victoire, défaite ou poursuite

ÉTAT

Anonyme : session Django.
Connecté : SaveGame + session.
État sérialisé en JSON.
Partie terminée non rechargeable.

RÈGLES MÉTIER

Combat par session.
Soins et dégâts côté serveur.
Propriété des sauvegardes.
Échec IA sans progression persistée.

Les tests portent sur les transitions et les droits, au-delà du rendu des pages.

Protéger les requêtes et les données du joueur

Des contrôles concrets, avec des limites de production identifiées.

REQUÊTES

CSRF sur les POST navigateur.
Taille JSON limitée.
Quota sur les routes IA.
Contenu dynamique via
textContent.

PROPRIÉTÉ / SECRETS

Filtres par utilisateur.
État de combat isolé.
Secret et hôtes de production via
environnement.

LIMITES

Audit dépendances à compléter.
Cache partagé pour plusieurs
instances.
Revue des appels longs et de la
concurrence du graphe.

Les protections du navigateur ont évolué : le code courant prime sur les anciennes notes.

Rendre les interactions perceptibles et maîtrisées

Améliorations présentes ; validation manuelle encore nécessaire.

INTERACTIONS

Annonces de statut dynamiques.
Onglets avec sémantique ARIA.
Focus après mise à jour.
Barres de vie exposées comme progression.

VÉRIFICATION MANUELLE

Parcours au clavier.
Contrastes et zoom 200 %.
Lecteur d'écran.
Qualité des textes FR / EN.

SOBRIÉTÉ

Appels de jeu déclenchés par action.
Réutilisation du retrieval.
Images locales.
Poids et consommation à mesurer.

La présence d'attributs ARIA ne suffit pas à valider l'accessibilité réelle.

44 tests ciblés réussis sur le bloc applicatif

Vérifications locales du 14 septembre 2026.

LOT EXÉCUTÉ

Combat et isolation.
Sécurité des requêtes.
Templates et parcours simulé.
Sauvegardes, personnages,
métriques.

MIGRATIONS

`makemigrations --check`
`--dry-run`

Aucun changement détecté.
Aucune migration créée.

CI VERSIONNÉE

PR, push main / master, manuel.
Lint, checks, tests et couverture.
Dépendances verrouillées.
Résultats distants à joindre.

44 tests ne désignent ni la suite complète ni un pourcentage de couverture.

Construire, vérifier et conserver le même artefact

Le conteneur constitue l'unité de livraison prévue.

BUILD / SMOKE

Dockerfile Python 3.12.
Installation verrouillée.
Démarrage Gunicorn.
Image taguée SHA, test /health
puis export tar en CI.

EXPLOITATION

Configuration par environnement.
Données persistantes à prévoir.
Migrations au démarrage local.
Étape de release à contrôler en
production.

PROMOTION / RETOUR

Déployer le digest vérifié.
Conserver l'image précédente.
Contrôler la compatibilité DB.
Rejouer santé et parcours.

Le job staging est un placeholder : il ne déploie pas encore l'application.

Montrer le parcours, puis les garanties

Séquence préparée : 1 min 40, dans un environnement de démonstration.

00:00 → 00:35

Ouvrir une aventure et un personnage déjà prêts.
Afficher récit et choix.
Exécuter une action de jeu.

00:35 → 01:10

Montrer une sauvegarde du compte de test et sa reprise.
Présenter le résultat du test de propriété.

01:10 → 01:40

Afficher les résultats locaux et le workflow.
Nommer les preuves manquantes.
Introduire le cas d'incident E5.

Prévoir une capture de secours ; annoncer toute simulation utilisée.

Surveiller et résoudre un incident technique

Cas étudié : interférence entre deux combats dans un même processus.

SURVEILLER

Métriques, journaux et règles d'alerte du projet.
Distinguer fonctionnement technique et intégrité métier.

REPRODUIRE

Ancien code extrait de Git.
Deux états A et B.
Imbrication déterministe.
Aucun incident de production supposé.

VÉRIFIER

Comparer avant / après.
Identifier le correctif versionné.
Expliquer les limites et les suites opérationnelles.

E5 : 10 minutes ; dossier technique complémentaire joint au support.

Observer le parcours complet du joueur

Django /metrics → Prometheus → Grafana ; journaux de services en console.

APPLICATION

Démarrages, tours et résultats.
Combats et actions.
Délai navigateur jusqu'au retour de
choix utilisables.

SERVICES IA

Opérations, tentatives et retries.
Durée logique incluant reprises.
Échecs, sorties invalides et RAG.
Coût estimé si tarifs configurés.

JOURNAUX

Date, niveau, logger, message.
Loggers agents et retrieval.
Erreurs contextualisées.
Contenu à revoir avant partage.

Les métriques évitent les identités ; les messages et traces doivent aussi être contrôlés.

Les alertes techniques ne détectent pas tout

Règles existantes dans `monitoring/prometheus/alerts.yml`.

INDISPONIBILITÉ

Plus de 2 opérations indisponibles sur 10 min.
Condition maintenue 2 min.
Signal pour diagnostic fournisseur.

LATENCE / QUALITÉ

p95 > 30 s sur fenêtre 15 min, pendant 10 min.
Sortie invalide : fenêtre 15 min, maintien 1 min.

ANGLE MORT MÉTIER

Un combat peut renvoyer HTTP 200 tout en modifiant le mauvais état.
Aucune règle actuelle ne détecte cet invariant.

Le récepteur Alertmanager est vide : aucune réception externe démontrée.

L'ancienne version attaque le mauvais monstre

Scénario contrôlé : chaque monstre démarre à 8 PV ; dégâts fixés à 3.

1 / RESTAURER A

Le processus charge le joueur A et son monstre dans des variables globales.

2 / RESTAURER B

Avant l'attaque de A, B charge son combat.
Les mêmes variables globales pointent maintenant vers B.

3 / REPRENDRE A

L'action destinée à A utilise l'état global de B.
Monstre A : 8 PV.
Monstre B : 5 PV.

Attendu : A perd 3 PV ; observé avant correctif : B perd 3 PV.

Un état global partage une identité implicite

La sélection du combat dépendait du dernier appel de restauration.

AVANT

player et current_monster au niveau du module.
restore_combat remplace leurs références.
player_action les lit implicitement.

MÉCANISME

Chaque session web possède ses données, mais la fonction métier réutilise une mémoire de processus partagée.

CONSÉQUENCE

Une interruption entre restauration et résolution peut cibler un autre combat.
Les compteurs HTTP ne voient pas cette erreur métier.

L'isolement de la session web doit être conservé jusque dans les fonctions métier.

Passer le combat explicitement aux fonctions

Correctif versionné le 3 septembre 2026 : commit 8a3cf39.

OBJET DÉDIÉ

CombatSession regroupe joueur, monstre, journal et défense.
Une instance liée à l'état traité.

APPELS EXPLICITES

```
restore_combat_session(...)
↓
resolve_player_action(session,
action)
↓
resolve_monster_attack(session)
```

MISE À JOUR LOCALE

GameEngine récupère joueur et monstre dans cette instance, puis met à jour son état de partie.

La résolution du combat Django ne dépend plus du dernier combat restauré globalement.

Le même scénario touche désormais le bon état

Résultats vérifiés le 14 septembre 2026.

AVANT CORRECTIF

Monstre A : 8 → 8 PV.
Monstre B : 8 → 5 PV.

Le mauvais combat est modifié.

CODE ACTUEL

Monstre A : 8 → 5 PV.
Monstre B : 8 → 8 PV.

L'action reste dans la session A.

RÉGRESSION

Test d'isolation Django réussi.
44 tests applicatifs ciblés réussis.
Cas absent / invalide et reprise de combat également exercés.

L'imbrication est prouvée ; la montée en charge et les requêtes simultanées restent à tester.

Relier le correctif à l'exploitation

Une résolution documentée et des actions opérationnelles encore à compléter.

PREUVES DISPONIBLES

Scénario, cause et correctif.
Commit historique identifié.
Reproduction avant / après.
Test de non-régression et commandes.

À JOINDRE

Ticket dans l'outil de suivi.
Chronologie de découverte réelle.
Alerte reçue en environnement test.
Déploiement du correctif et contrôle après livraison.

PRÉVENIR

Tester les invariants métier.
Revoir les autres états partagés.
Tester concurrence et charge.
Mettre à jour le guide de diagnostic.

Le dossier E5 distingue les faits historiques, la reconstruction et les actions proposées.

Rejouer les preuves sans toucher au jeu

Depuis la racine du dépôt ; aucune génération réelle requise.

AVANT / APRÈS

```
uv run python  
certif/presentation-bloc3/  
reproduce_incident.py
```

Ancien code lu via Git.

TEST D'ISOLATION

```
uv run python  
src/django/manage.py test  
game.tests.CombatEngineTests  
--noinput
```

MIGRATIONS

```
uv run python  
src/django/manage.py  
makemigrations --check  
--dry-run
```

Les commandes exactes et le lot complet figurent dans [preuves/verification.md](#).

Retrouver les pièces du dossier

Documentation applicative sous docs/block3/.

CADRAGE / RÉALISATION

C14 : besoins et traçabilité.
C15 : modèles et décisions.
C16 : backlog et revues.
C17 : code, qualité et sécurité.

QUALITÉ / LIVRAISON

C18 : tests et workflow CI.
C19 : Dockerfile, artefact et
procédure de promotion.
Preuves distantes à ajouter.

EXPLOITATION

C20 : monitoring, règles et logs.
C21 : dossier d'incident, correctif
Git et reproduction.
Ticket et déploiement à ajouter.

Les statuts des anciennes notes peuvent être en retard sur le code : dater les preuves.

Expliquer les frontières de la preuve

Trois questions probables du jury.

EST-CE DÉPLOYÉ ?

Le packaging et la procédure existent. Le job staging reste à compléter ; aucune mise en production prouvée ici.

EST-CE ACCESSIBLE ?

Des améliorations et tests de structure existent. Clavier, contraste, zoom et lecteur d'écran restent à auditer.

INCIDENT DÉTECTÉ COMMENT ?

Défaut historique reconstruit par imbrication contrôlée. Aucune alerte de production ni détection automatique de ce défaut n'est revendiquée.

Présenter exactement ce qui a été observé, testé et livré.